ALT Codebase Architecture Analysis

---

1. Project Overview

The ALT system is a .NET 8 monorepo comprising 7 projects and 97 controllers. It is the central manufacturing execution layer at Maryville DMTN, handling permissive process control, part traceability, operator HMI, and operational telemetry for 250+ lines and ~3,500 connected machines.

Run locally with .NET Aspire — a single `dotnet run` in the Aspire host fires all services simultaneously. ADO pipelines deploy from the same repo.

Project	Controllers	Criticality	Purpose
ALT Core API	26	Production Critical	Core PLMS business logic — R2P, StoreResults, all permissive control
ALT UI API	20	Mission Critical	Pass-through proxy to ALT Core for engineer/team leader UI
NA Data API	20	Mission Critical	IoT data CRUD, MQTT publishing, Event Hub bridge
Flux	15	Production Critical	Operator HMI — scan parts, run production, line-side client
ALT Receiving	8	Mission Critical	Reel labeling, receiving, kanban scanning
Jarvis Core	6	Mission Critical	LETS integration, Maximo database connection
PLMS Ship API	4	Production Critical	Ships completed parts to NASCS/ERP (CIGMA)

Criticality tiers (client-defined):

- **Production Critical** — line stops immediately if this fails
- **Mission Critical** — important but line can continue briefly, data can be backfilled

---

2. Controllers and Endpoints

PLMS Controller — `api/PLMS` (Production Critical)

The most critical controller in the system. Every machine interaction on every line passes through here.

R2P Endpoints (7 variants)

Endpoint	Method	Line	Purpose
`RequestToProcessPost`	POST	58	Primary — takes `R2PModel` DTO
`RequestToProcess`	GET	118	Query param variant
`RequestToProcessFlowControl`	GET	179	Kanban-aware variant
`RequestToProcessFLUX`	GET	220	Flux HMI variant
`RequestToProcessWithSerialReturn`	GET	261	Returns serial on pass
`RequestToProcessPanel`	GET	315	Panel board arrays
`RequestToProcessThenStoreResultsPanel`	GET	375	Combined R2P + Store in one call

StoreResults Endpoints (14 variants)

Endpoint	Method	Line	Notes
`StoreResults`	GET	488	Basic single serial
`StoreResultsUSXPost`	POST	1075	**Primary POST — hot path**
`StoreResultsUSX`	GET	600	Multi-serial (us1, us2, usp)
`StoreResultsUSXPlusDigitalAssets`	GET	823	Includes digital assets
`StoreResultsUSXWithData`	GET	937	With extended data
`StoreResultsUSXAutomated`	GET	1006	Kanban-based automation
`StoreResultsUSXPost_SA_Complete`	POST	1175	Sub-assembly complete
`StoreResultsUSXPostSMD`	POST	738	SMD line variant
`StoreResultsPanelArray`	GET	535	Panel array
`StoreResultsPanel`	GET	1240	Panel
`StoreResultsSixPanelArray`	POST	1590	6-up panel
`StoreResultsJSON/{div}/{loc}/{line}/{proc}/{mach}`	POST	1381	Route-param variant
`StoreResultsArray/{div}/{loc}/{line}/{proc}/{mach}`	POST	1423	Array variant

> The large number of variants reflects 30 years of organic growth — different machine builders and process types required different payload shapes. The hot path for the POC is `RequestToProcessPost` and `StoreResultsUSXPost`.

LETS Controllers (Mission Critical)

Controller	Endpoints	Focus
`LETSFrontLineLeaderController`	95	Shifts, production counts, manning, OEE
`LETSLineEventTrackingController`	61	Machine states, faults, alerts
`LETSFrontLineLeader5MController`	12	5-minute event tracking

---

3. Data Flow — HTTP Request to Stored Procedures

All SP calls route through `PLMSLookupContext` via `Database.ExecuteSqlRawAsync()`. No raw ADO.NET — all SP execution is abstracted through EF Core's raw SQL execution.

R2P Call Chain

```
PLMSController.RequestToProcess()
  → PLMS_BO.RequestToProcess()
    → RequestToProcessModel.CallRequestToProcess()
      → RequestToProcess_DAL.RequestToProcess()
        → PLMSLookupContext.RequestToProcess()
          → [dbo].[requestToProcess]
```

StoreResults Call Chain

```
PLMSController.StoreResultsUSXPost()
  → PLMS_BO.StoreResultsUSX()
    → StoreResultsUSXModel.CallStoreResultsAsync()
      → StoreResults_DAL / StoreResults2_DAL / StoreResults3_DAL
        (branches on serial count — 1, 2, or 3 serials being married)
        → PLMSLookupContext.StoreResults / storeResults2 / storeResults3
          → [dbo].[storeResults] / [storeResults2] / [storeResults3]
```

Ship Call Chain

```
PLMSController → PLMS_BO.ShipUSX()
  → ShipUSXModel → ShipUSX_DAL
    → PLMSLookupContext.ShipUSX()
```

```
    → [dbo].[ship]
    → Ship API → NASCS/CIGMA (ERP)
```

All Stored Procedures

Stored Procedure	Purpose
`[dbo].[storeResults]` / `storeResults2` / `storeResults3`	Store part results (branches on serial count)
`[dbo].[requestToProcess]`	Core R2P permissive check
`[dbo].[requestToProcess_FlowControl]`	Kanban-aware R2P
`[dbo].[requestToProcess_FLUX]`	Flux HMI R2P
`[dbo].[ship]`	Mark part as shipped, fire to ERP
`[dbo].[verifyFormat]`	Serial format validation
`[dbo].[insert_FA_RepairStation]`	Final assembly repair station record
`[dbo].[PostSACompleteStatus]`	Sub-assembly complete status
`[dbo].[procCreateProcessTable]`	Create per-line data process table

> Hank is actively refactoring these stored procedures to EF Core ORM calls as part of the AIO edge migration. The edge-db (PostgreSQL/MySQL) will not support these SPs — the refactor is a prerequisite for edge deployment.

---

4. MQTT Architecture

Broker

- **Current:** `20x-lnxmqtt-cl.dmtnprod.lan` — VM-based clustered Linux MQTT broker in IDMZ
- **Target:** AIO MQTT broker VIP on K3s cluster
- **Migration:** Configuration-only change in `MQTTClientService.cs` — no code changes required. All services read the broker URL from config.
- **Library:** MQTTnet via `MQTTClientService.cs`

Published Topics

Topic	Publisher	Trigger
`OEE/{Div}/{Loc}/{Line}`	LETSStateService	Timer (downtime threshold)
`LineStateDown/{Div}/{Loc}/{Line}`	LETSStateService	Line state transition
`LineState/{LineID}/{Process}`	CurrentLineStateService	Continuous 1s loop
`HourlyAttainment/{Div}/{Loc}/{Display}`	HourlyAttainmentService	Hourly schedule
`HourlyAttainmentActTwo/{Line}`	HourlyAttainmentService	Hourly schedule
`ProcessRequest/{Div}/{Loc}/{Line}/{Proc}/{Mach}`	NA_Data_Utility	HTTP POST
`Traceability/{Div}/{Loc}/{Line}/{Proc}/{Mach}`	NA_Data_Utility	HTTP POST
`TraceabilityV2/{Div}/{Loc}/{Line}/{Proc}/{Mach}`	NA_Data_Utility	HTTP POST (full data)
`KanbanShipStatus/{Line}`	KanbanStorageUtility	Kanban API response

Subscribed Topics

Topic	Subscriber	Handler
`LineState/{Div}/{Loc}/#`	LETSStateService	SetNewLineState()
`Traceability/{Div}/{Loc}/#`	LETSStateService + EventHub	UpdateMachineCycle()
`LineSettings/{Div}/{Loc}/#`	LETSStateService	UpdateLineSettings()

Topic	Subscriber	Handler
`UpdateLineState/#`	CurrentLineStateService	UpdateCurrentState()
`TagLineState/#`	CurrentLineStateService	RetagLineState()
`ReloadLineInfo/#`	CurrentLineStateService	RefreshLineConfigOnly()
`KanbanShipStatus/#`	MQTTtoEventHub	ForwardKanbanShipStatus()

Topic Naming Convention

`{Div}/{Loc}/{Line}/{Proc}/{Mach}` — Division / Location / Line / Process / Machine

For Maryville DMTN: `TN/DMTN/Line-103/...`

This is the confirmed topic structure for AIO MQTT broker configuration and AIO dataflow subscriptions. The open LLD item "topic hierarchy undefined" is now closed.

---

5. Database Architecture

8 DbContext Classes → 6 Physical Databases

Context	Physical DB	Server	Access Pattern	Edge-needed?
ALTDataContext	PLMS	AG cluster	EF Core LINQ (100+ entities)	Yes
PLMSLookupContext	PLMS	AG cluster	Mixed — LINQ + SPs	Yes
PLMSDataContext	PLMS	AG cluster	SPs via FromSqlRaw	Yes
IoTContext	PLMS	AG cluster	EF Core LINQ (26 DbSets)	Yes
CellCon_DataContext	PLMS	AG cluster	FromSqlRaw only	Yes
JarvisDbContext	Jarvis	10.73.120.189	EF Core LINQ (40+ DbSets)	No — stays on-prem
AltReceivingContext	ReelLabel	dmtn-altdb.dmtnprod.lan	Mixed	No — out of POC scope

Context	Physical DB	Server	Access Pattern	Edge-needed?
MMSContext	MMS	10.73.40.238	EF Core LINQ	No — stays on-prem

Maximo DB: 10.72.73.34 (dmtnmax) — connected via Jarvis API. Already live today. Connection string must be updated when Maximo moves to Azure NA-wide.

Edge-DB Scope

Only PLMS data needs to be replicated to the edge-db (PostgreSQL/MySQL). Jarvis, MMS, Maximo, ReelLabel remain pointing at on-prem/cloud servers. This significantly reduces the edge-db footprint — it holds only what the line needs for R2P, StoreResults, and recipe lookup.

Stored Procedure vs EF Core

Stored Procedures (migration required for edge):

- PLMSLookupContext: all R2P and StoreResults hot-path operations
- PLMSDataContext: `procDataProcessQuery`, `getPLMSDatabaseNames`, `procArchiveQuery`
- CellCon_DataContext: `GetBySerial`, `GetShipHistories`, `GetFilteredRecords`
- AltReceivingContext: `ReelLabelReport`, `sp_LabelReprint`

Pure EF Core LINQ (edge-ready):

- IoTContext — all 26 entities
- JarvisDbContext — all mutations via MutatorExtensions
- ALTDataContext — 100+ entities via QueryExtensions

---

6. OTEL and Logging

OpenTelemetry — Already Implemented in 11 Services

> **Key finding**: OTEL migration was incorrectly scoped as a future code change. The instrumentation is already built. Deploying the OTEL Collector and pointing it at Azure Monitor is the only remaining step.

Each service has `InstrumentationConfig.cs` defining:

- `ActivitySource` + `Meter` + `Counter<long>("app.request_counter")`

- Registered in `Program.cs` with tracing, metrics, and logging OTLP exporters

Services with full OTEL instrumentation: Flux, ALT Core API, PLMS Ship API, ALT UI API, ALT Receiving API, Jarvis API, IOT Email API, IOT Image API, IOT WMS API, AndonClient, MQTTtoEventHub.

ClientLogger — Custom Logging Bridge

`ClientLogger.cs` bridges `ILogger` and HTTP POSTs to IOT Logger API (port 8000). Methods: `Error`, `Info`, `Warn`, `Debug`. Supports `OTEL_Only` mode — when set, skips all IOT Basics HTTP calls and uses OTEL directly. This means switching to OTEL-only logging requires only a configuration flag change in 11 services, not a code change.

Registered as singleton in all 11+ services.

NLog — Containerization Blocker

NA_Data_API uses NLog writing to `c:\temp\logs\` — a hardcoded Windows filesystem path. This will fail immediately in a Linux container on K3s. Must be replaced with the OTEL stack before NA_Data_API can be containerized. This is a pre-containerization prerequisite to flag to Rob and Hank.

NextGen / Aspire

`ALT.ServiceDefaults/Extensions.cs` adds:

- Health checks at `/health` and `/alive` — used by K3s liveness and readiness probes
- Conditional OTLP export based on environment

IoT Basics — Port 8000

All 11+ services log to IOT Logger API on port 8000 via `ClientLogger`. The `alt-system` NetworkPolicy must explicitly allow all ALT pods to reach port 8000 on the IoT Basics pod. Without this rule all logging calls will fail silently.

---

7. Configuration Model

Hierarchy

```
Plant (PlantCode = "TN")
  └── Division (LineDivision)
        └── Line (lookup_searchdb)
              ├── LineSetupStates   — manning and staffing
```

```
├── LineShifts        — shift start/end times
├── ProcessFlow       — routing definition
│     └── FlowSequence — step ordering
└── SetupBws          — recipe (85 columns per row)
      — per part × process with validation rules,
        vision config, robot config, printer config,
        kanban logic, skip logic
```

Key Config Entities

Model	Table	Cache TTL	Purpose
`LookupSearchDB`	`lookup_searchdb`	5 min	Line metadata — DB name, dept, connection strings
`LineContext`	In-memory record	5 min	Assembled runtime line context
`ProcessFlow`	`ProcessFlow`	—	Process routing definitions
`FlowSequence`	`FlowSequence`	—	Step ordering within a flow
`SetupBws`	`setup_bws`	5 sec	Recipe — 85 columns including validation rules, robot config, vision config, kanban, skip logic
`SetupContext`	In-memory record	5 sec	Assembled runtime recipe context

Access Patterns

- **Line lookup:** `ILineContextService.GetLineContextAsync(lineName)` → queries `lookup_searchdb` → cached 5 min → returns `LineContext`
- **Recipe assembly:** `ISetupService.BuildContextAsync(lineCtx, process, part, serial, type)` → loads `SetupBws` rows → parses `Extra_vars` → validates → cached 5 sec → returns `SetupContext`
- **Background refresh:** `LineRefreshService` pre-warms line cache every 30 min
- **Environment selection:** `ALT_CORE_ENV` env var selects config section (URLS / DEVURLS / STAGURLS / plant-specific)

> **LLD implication:** The edge-db schema must support the lookup patterns `ILineContextService` and `ISetupService` expect. A new `ALT_CORE_ENV` value is needed

> for the edge deployment configuration section — do not reuse existing URLS/STAGURLS values.

---

8. Cross-Service Dependencies

```
                    +------------------+
                    |  NASCS / CIGMA   |  (ERP — external, OAuth2)
                    +--------+---------+
                             |
+-----------+    +-----------v-----------+    +--------------+
|   Flux    +--->+    ALT Core API       +<---+  Ship API    |
| (60+ calls)|   |   (central hub)       |    |              |
+-----------+    +--+-----+------+-------+    +--------------+
      |             |     |      |
      |        +----v-+ +-v---+ +--> +------------+
      |        | MQTT | |NA   |      | Jarvis API |
      |        | Brkr | |Data |      | (LETS +    |
      |        +--+---+ |API  |      |  Maximo)   |
      |           |     +--+--+      +------+-----+
      |           |        |                |
      |    +------v--+   +-v----------+  +--v-------+
      |    |EventHub |   |IOT Logger  |  | Maximo   |
      |    |(cloud)  |   |API :8000   |  | DB       |
      |    +---------+   +------------+  +----------+
      |
+-----v------+
|  ALT UI    +---(30+ calls)---> ALT Core API
|  (20+ to   +---(20+ calls)---> Jarvis API
|  Jarvis)   |
+------------+
```

Key Call Volumes

Caller	Target	Calls	Protocol
Flux	ALT Core API	60+	HTTP (NamedHttpClient: AltCoreApi)
Flux	Jarvis API	20+	HTTP (JarvisPassThroughService)
Flux	ALT Receiving	10+	HTTP (AltReceivingApi)

Caller	Target	Calls	Protocol
Flux	DWAM Manager	—	HTTP (DwamMgrApi / IVIDwamMgrApi)
ALT UI	ALT Core API	30+	HTTP pass-through
ALT UI	Jarvis API	20+	HTTP (direct HttpClient)
Ship API	NASCS/CIGMA	—	HTTP OAuth2
All services	IOT Logger API	2× per call	HTTP POST to port 8000
ALT Core / Jarvis / CurrentLineState	MQTT broker	Event-driven	MQTTnet

9. AIO Migration Notes

What changes at deployment time

Item	Change type	Owner
MQTT broker URL → AIO VIP	Config only	Insight / ALT team
`ALT_CORE_ENV` → new edge value	Config only	Insight / ALT team
NLog in NA_Data_API → OTEL	Code change	ALT dev team
Stored procedures → EF Core ORM	Code change (Hank's workstream)	ALT dev team
OTEL Collector deployment	Infrastructure	Insight
IOT Basics port 8000 NetworkPolicy	Infrastructure	Insight
Maximo connection string → Azure URL	Config (when Maximo migrates)	ALT team + Maximo team

What does NOT change

- All controller endpoint paths — changing these would break all line integrations
- PLMS endpoint naming — legacy name kept intentionally

- MQTTnet library — same library, different broker URL
- `ClientLogger` interface — `OTEL_Only` flag enables OTEL without API refactor
- Health check endpoints `/health` and `/alive` — already present, used by K3s probes

Pre-containerization checklist

- [ ] NA_Data_API: replace NLog with OTEL (`c:\temp\logs\` path will fail on Linux)
- [ ] All services: set `ALT_CORE_ENV` to new edge config section
- [ ] All services: update MQTT broker URL to AIO VIP
- [ ] Confirm `ClientLogger` `OTEL_Only` mode enabled in edge environment
- [ ] Validate `/health` and `/alive` endpoints respond correctly in container
- [ ] Confirm Jarvis DB connection (10.73.120.189) reachable from cluster NIC 2